NP-completeness solutions

34-E01 Nondeterministic paths and verifiers

An NTM first guesses y of length at most q(|x|), then deterministically runs V(x,y). Each path is polynomially long. This does not enumerate all exponentially many witnesses in deterministic polynomial time. For x in L, completeness supplies an accepting path. For x outside L, soundness rejects every path.

Conversely, given a polynomial-time NTM M, record the nondeterministic choices along an accepting path as certificate y. The fixed machine has bounded branching, and the polynomial time bound limits the number of choices, so y has polynomial length. A deterministic verifier simulates M along those choices and accepts exactly when the simulated path accepts. Hence acceptance is equivalent to existence of that choice string.

The two definitions coincide. For **each** x, membership is equivalent to existence of a bounded accepted certificate. It is not true that every x has an accepting certificate: instances outside the language have no accepting certificate.

34-E02 Bit length and pseudopolynomial subset-sum

Every ai has $\Theta(m)$ bits, and t has m+1, so $N = \Theta(m^2)$. Full-table DP costs $\Theta(m2^m) = 2^{\Theta(\sqrt{N})}$. A fixed polynomial upper bound $CN^c = O(m^{2c})$ would bound an exponential by a fixed-degree polynomial, a contradiction. This concerns the work of this full-table algorithm, rather than proving a lower bound for every possible algorithm from one O upper bound.

A fixed choice such as n=200,B=2^80 is one finite instance and proves no asymptotic claim. In this family every ai is $\Theta(2^m)$, so fully unary input length is $\Theta(m2^m)$ and the DP cost is polynomial in that length. Generally, unary input length is at least n+t and $nt \leq ((n+t)/2)^2$. Merely knowing ai≤B does not mean every input has unary length $\Theta(nB)$.

34-E03 Full three-CNF-SAT to clique construction

The groups are C1={x1,¬x2,x3}, C2={¬x1,x2,¬x3}, C3={x1,x2,x3}. There are no edges within groups and all noncomplementary cross-group edges. Assignment x1=false,x2=false,x3=true satisfies F. Choose C1.x3, C2.¬x1, C3.x3. They are all true and pairwise noncomplementary, giving a 3-clique. In contrast, (true,false,true) falsifies the entire second clause and is not a satisfying witness.

NP-completeness solutions (continued)

For any satisfying assignment, choose one true occurrence in each group. Consistency prevents complementary pairs, so every selected pair is adjacent and gives a k-clique. Conversely, a size-3 clique must choose exactly one vertex per group. Its edges forbid complementary chosen literals. Assign their variables to make them true and assign remaining variables arbitrarily. Every clause has a chosen true literal, proving satisfiability.

There are 3k vertices and $O(k^2)$ word comparisons. Intern names as n IDs first. A bit-cost bound is $O(|F| \log |F| + k^2 \log(n+k))$, still polynomial. The construction and both implications complete the reduction.

34-E04 Clique and vertex cover in both directions

In G, {1,2} is a 2-clique. Gbar has only edge 13, so {1,2} is independent there. Its complement {3} covers the only edge and has size n-k=1.

For a clique C in G, every pair has a G edge and therefore no Gbar edge, so C is independent in Gbar. If S is independent in Gbar, no Gbar edge has both endpoints in S. Thus $V \setminus S$ covers every edge. Conversely, a cover C0 forbids an edge entirely in $V \setminus C0$, making that complement independent in Gbar and therefore a clique in G.

A size-k clique corresponds to a size-(n-k) cover. With at-least and at-most thresholds, restrict a larger clique or enlarge a smaller cover as needed. The reductions are $(G,k) \mapsto (\overline{G}, n-k)$ and $(H,\ell) \mapsto (\overline{H}, n-\ell)$. Both build the complement in $O(n^2)$ pair checks, explaining both thresholds and directions.

34-E05 Hamiltonian cycle to TSP decision

For K3, all original edges receive weight 1 and B=3. Tour 1-2-3-1 costs 3. Generally, for n≥3, build Kn with weight 1 on original edges and 2 on nonedges, and set B=n. An HC uses n original edges and gives cost n. Conversely, a tour has exactly n edges, each at least 1. Cost at most n forces all weights to be 1, so every edge is original and the tour is an HC. Handle n<3 by a fixed no target under the chapter's simple undirected-cycle convention.

A TSP-decision certificate is a permutation of the n vertices. Verify each vertex exactly once, closure, complete-graph edges, and total cost at most B in polynomial time. Optimization asks for minimum cost and possibly a tour. Decision only answers whether the threshold is feasible. Its NP-complete language is not the same output type as an optimal-tour construction.

NP-completeness solutions (continued)

34-E06 Transitivity and reduction direction

Let h(x)=g(f(x)). If x is in A, f's equivalence gives f(x) in B, then g's gives h(x) in C. Conversely, h(x) in C gives f(x) in B and then x in A. If f takes pf(n) time, its output length is at most pf(n). Running g then takes at most pg(pf(n)), so total $O(p_f(n) + p_g(p_f(n)))$ is polynomial. Thus A$\leq$pC.

If C is NP-hard and C$\leq$pL, any A in NP reduces to C and then L, proving L NP-hard. L$\leq$pC only says that L is no harder than C under the reduction. Hardness does not transfer backward. Separately proving L in NP is necessary to conclude NP-completeness.

34-E07 Gate constraints need both implications

For OR use $(u \vee \neg a) \wedge (u \vee \neg b) \wedge (\neg u \vee a \vee b)$. For NOT use $(v \vee a) \wedge (\neg v \vee \neg a)$. For AND use $(\neg z \vee u) \wedge (\neg z \vee v) \wedge (z \vee \neg u \vee \neg v)$, plus unit clause z. Input a=0,b=1 gives u=v=z=1 and satisfies every clause.

With z forced to 1, the omitted clause $(z \vee \neg u \vee \neg v)$ is already true. Its omission creates no immediate local error in this particular example. The remaining two clauses still force u=v=1. That special case cannot justify omission in general.

For a counterexample, let z=a AND b, w=NOT z, fix a=b=1, and require w=1. The real circuit forces z=1,w=0 and is unsatisfiable. Incomplete AND constraints z⇒a and z⇒b allow forged values a=b=1,z=0,w=1. They satisfy those constraints, the complete NOT gadget, and the output requirement. The missing reverse implication permits a false zero gate output. Recovery of true wire values in topological order requires complete gate equivalences.

34-E08 Local and global Hamiltonian-cycle gadgets

Vertices a2,a5,b2,b5 have degree two and force eight adjacent chain edges. Then a3-b1 and a4-b6 have indicator 1-x, while a1-b3 and a6-b4 have 1-y. Ports a1,a6 have internal degree 2-y and external degree y. Ports b1,b6 have external degree x.

For x=y=0, the cycles are a1-a2-a3-b1-b2-b3-a1 and a4-a5-a6-b4-b5-b6-a4. All vertices already have degree two and no external port is available. They cannot connect to a selector and cannot belong to the global single cycle.

Let U=W_uv have a side u and b side v. Let V=W_vw have a side v and b side w. The full cycle is

NP-completeness solutions (continued)

```
s1,
U.b1,U.b2,U.b3,U.a1,U.a2,U.a3,U.a4,U.a5,U.a6,U.b4,U.b5,U.b6,
V.a1,V.a2,V.a3,V.b1,V.b2,V.b3,V.b4,V.b5,V.b6,V.a4,V.a5,V.a6,
s1
```

In U, x=1,y=0 gives the full v-side path. In V, x=0,y=1 gives the full v-side path. Within each row, every adjacency is among its fourteen internal edges. U.b6-V.a1 is the original vertex v's chain link. The first and final edges are selector links. The two gadgets supply 24 distinct vertices, and s1 supplies the twenty-fifth, with no repeated vertex before closure.

The selected side's ports must connect externally as a pair. A path cannot enter on u's side and leave through v's chain. Local modes and the global selector-chain structure together decode a cover. A local path alone is insufficient for the full reduction.

34-E09 No-carry reduction and reverse decoding

In column order x1,x2,x3,C1,C2, the numbers are T1=10010,F1=10001; T2=01001,F2=01010; T3=00111,F3=00100. Slack numbers are 00010,00020,00001,00002, and target is 11144.

Assignment (0,1,1) selects F1+T2+T3=10001+1001+111=11113. Clause true counts are 1 and 3. Add S1+S1′+S2=10+20+1 to reach 11144. Assignment (0,1,0) selects F1+T2+F3=11102. The first clause contributes zero and its two slack numbers add at most 3, so it cannot reach 4. Other columns cannot carry into this deficit.

For every subset, each variable column is at most 2. A clause receives at most three literal 1s plus slack 1+2, totaling at most 6<10. Thus no carry occurs for any candidate. Reaching the target forces exactly one Ti or Fi per variable, and at least one true literal in each clause since slack alone is at most 3.

There are 2n+2k numbers, each with n+k decimal columns and $O(n+k)$ binary bits, so total output is $O((n+k)^2)$ bits. Exponentially large values do not imply exponentially long output. The no-carry statement must hold for all candidates to justify reverse decoding.

NP-completeness solutions (continued)

Verification code appendix: clique_reduction.py

```
"""Check both directions of the Chapter 34 3-CNF-SAT to CLIQUE reduction."""

from itertools import combinations, product

# Positive integer means x_i; negative integer means not x_i.
formula = (
    (1, -2, 3),
    (-1, 2, -3),
    (1, 2, 3),
)


def satisfiable(clauses):
    variables = sorted({abs(literal) for clause in clauses for literal in clause})
    for bits in product((False, True), repeat=len(variables)):
        assignment = dict(zip(variables, bits))
        if all(any(assignment[abs(lit)] == (lit > 0) for lit in clause) for clause in
clauses):
            return True
    return False


def build_graph(clauses):
    vertices = [(row, column, literal) for row, clause in enumerate(clauses) for column,
literal in enumerate(clause)]
    edges = set()
    for left, right in combinations(vertices, 2):
        if left[0] == right[0]:
            continue
        if left[2] == -right[2]:
            continue
        edges.add(tuple(sorted(((left[0], left[1]), (right[0], right[1])))))
    return vertices, edges


def has_k_clique(vertices, edges, k):
```

NP-completeness solutions (continued)

```
    for selected in combinations(vertices, k):
        if len({vertex[0] for vertex in selected}) != k:
            continue
        pairs = combinations(selected, 2)
        if all(tuple(sorted(((a[0], a[1]), (b[0], b[1])))) in edges for a, b in pairs):
            return True
    return False


def verify_equivalence(clauses):
    vertices, edges = build_graph(clauses)
    return satisfiable(clauses) == has_k_clique(vertices, edges, len(clauses))


clause_domain = tuple(product((1, -1, 2, -2), repeat=3))
examples = (
    formula,
    ((4, 4, 4), (-4, -4, -4)),
    (),
) + tuple(
    (left, right)
    for left in clause_domain
    for right in clause_domain
)
for example in examples:
    assert verify_equivalence(example), example

vertices, edges = build_graph(formula)
print(
    f"cases={len(examples)} vertices={len(vertices)} edges={len(edges)} "
    "satisfiable_iff_k_clique=verified"
)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP-completeness solutions (continued)

Verification code appendix: reduction_constructions.py

```
"""Executable checks for the explicit Boolean, HC, and subset-sum gadgets."""
from itertools import combinations, product


def eval_clause(clause, assignment):
    return any(assignment[abs(lit)] == (lit > 0) for lit in clause)


def gate_clauses(kind, z, u, v=None):
    if kind == 'and':
        return ((-z,u),(-z,v),(z,-u,-v))
    if kind == 'or':
        return ((z,-u),(z,-v),(-z,u,v))
    return ((z,u),(-z,-u))


def pad(clause, p, q):
    if len(clause)==3:
        return (clause,)
    if len(clause)==2:
        return (clause+(p,),clause+(-p,))
    return tuple(clause+(a*p,b*q) for a,b in product((1,-1),repeat=2))


def widget_edges():
    return ([(i,i+1) for i in range(5)]
            +[(i,i+1) for i in range(6,11)]
            +[(0,8),(2,6),(3,11),(5,9)])


def widget_modes():
    edges=widget_edges()
    ports={0,5,6,11}
    modes=[]
    for mask in range(1<<len(edges)):
        adjacency=[[] for _ in range(12)]
        for i,(u,v) in enumerate(edges):
            if mask>>i&1:
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP-completeness solutions (continued)

```
                adjacency[u].append(v)
                adjacency[v].append(u)
        if any(len(adjacency[v])!=2 for v in range(12) if v not in ports):
            continue
        if any(len(adjacency[v])>2 for v in ports):
            continue
        seen=set()
        endpoints=[]
        valid=True
        for start in range(12):
            if start in seen:
                continue
            todo=[start]
            component=[]
            while todo:
                u=todo.pop()
                if u in seen:
                    continue
                seen.add(u)
                component.append(u)
                todo.extend(adjacency[u])
            ends=[u for u in component if len(adjacency[u])<2]
            if len(component)==1 and start in ports:
                endpoints.append((start,start))
            elif len(ends)==2 and set(ends)<=ports:
                endpoints.append(tuple(sorted(ends)))
            else:
                valid=False
                break
        if valid:
            modes.append(tuple(sorted(endpoints)))
    return sorted(modes)


def numbers_for(clauses, n):
    k=len(clauses)
    def encode(digits):
        return int(''.join(map(str,digits)))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NP-completeness solutions (continued)

```
    values=[]
    for i in range(1,n+1):
        for literal in (i,-i):
            digits=[int(j==i-1) for j in range(n)]
            digits += [int(literal in clause) for clause in clauses]
            values.append(encode(digits))
    for j in range(k):
        for digit in (1,2):
            values.append(encode([0]*n+[digit if r==j else 0 for r in range(k)]))
    target=encode([1]*n+[4]*k)
    return values,target


def verify():
    gate_cases=0
    for kind in ('and','or','not'):
        for bits in product((False,True),repeat=3):
            assignment=dict(zip((1,2,3),bits))
            expected=bits[0] and bits[1] if kind=='and' else bits[0] or bits[1]
            if kind=='not':
                expected=not bits[0]
            result=all(eval_clause(c,assignment) for c in gate_clauses(kind,3,1,2))
            assert result==(bits[2]==expected)
            gate_cases+=1
    padding_cases=0
    for clause in ((1,),(1,2)):
        for bits in product((False,True),repeat=2):
            original=dict(zip((1,2),bits))
            extensions=[]
            for extras in product((False,True),repeat=2):
                assignment=original|dict(zip((3,4),extras))
                extensions.append(all(eval_clause(c,assignment) for c in pad(clause,3,4)))
            assert any(extensions)==eval_clause(clause,original)
            padding_cases+=1
    modes=widget_modes()
    assert modes==[((0,5),),((0,5),(6,11)),((6,11),)],modes
    clauses=((1,2,3),(-1,2,-3))
    values,target=numbers_for(clauses,3)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

NP-completeness solutions (continued)

```
    assert values==[10010,10001,1011,1000,110,101,10,20,1,2]
    assert target==11144
    chosen=[values[i] for i in (0,2,5,7,9)]
    assert sum(chosen)==target
    subset_cases=0
    satisfiable_masks=[]
    for mask in range(1<<len(values)):
        selected=[v for i,v in enumerate(values) if mask>>i&1]
        if sum(selected)!=target:
            continue
        assignment={}
        for i in range(3):
            assert ((mask>>(2*i))&1)+((mask>>(2*i+1))&1)==1
            assignment[i+1]=bool(mask>>(2*i)&1)
        assert all(eval_clause(c,assignment) for c in clauses)
        subset_cases+=1
        satisfiable_masks.append(mask)
    assert subset_cases>0
    for bits in product((False,True),repeat=3):
        assignment=dict(zip((1,2,3),bits))
        if not all(eval_clause(c,assignment) for c in clauses):
            continue
        base=sum(values[2*i+(not bits[i])] for i in range(3))
        slack=0
        for j,c in enumerate(clauses):
            count=sum(assignment[abs(lit)]==(lit>0) for lit in c)
            slack+=(4-count)*10**(len(clauses)-j-1)
        assert base+slack==target
    print(f'PASS gate_truth_rows={gate_cases} padding_assignments={padding_cases} '
          f'widget_edge_subsets={1<<14} widget_modes={len(modes)} '
          f'subset_masks={1<<len(values)} valid_subsets={subset_cases}')


if __name__=='__main__':
    verify()
```

NP-completeness solutions (continued)

Verification code appendix: reduction_depth.py

```
"""Witness construction for the chapter's concrete VC-to-HC reduction."""
from itertools import combinations
from pathlib import Path
import sys
sys.path.insert(0,str(Path(__file__).resolve().parent))
from reduction_constructions import widget_edges,numbers_for,gate_clauses,eval_clause

def cover_to_cycle(vertices,edges,cover):
    """Nonisolated simple graph, nonempty valid cover; returns graph and HC witness.

    This builds the witness from a supplied cover; it does not solve HC.
    """
    vertices=sorted(vertices);edges=sorted(tuple(sorted(e)) for e in
edges);cover=sorted(cover)
    if not cover or not all(u in cover or v in cover for u,v in edges):raise
ValueError('nonempty valid cover required')
    neighbors={u:sorted(v if u==w else w for w,v in edges if u in (w,v)) for u in vertices}
    if any(not neighbors[u] for u in vertices):raise ValueError('remove isolated vertices
first')
    graph={}
    def add(u,v):graph.setdefault(u,set()).add(v);graph.setdefault(v,set()).add(u)
    def node(u,v,i):
        a,b=sorted((u,v));return (a,b,i if u==a else i+6)
    for u,v in edges:
        for i,j in widget_edges():add((u,v,i),(u,v,j))
    for u in vertices:
        for a,b in zip(neighbors[u],neighbors[u][1:]):add(node(u,a,5),node(u,b,0))
        for j in range(len(cover)):
            add(('s',j),node(u,neighbors[u][0],0));add(('s',j),node(u,neighbors[u][-1],5))
    cycle=[]
    for j,u in enumerate(cover):
        cycle.append(('s',j))
        for v in neighbors[u]:
            a,b=sorted((u,v))
            if v in cover:indices=list(range(6)) if u==a else list(range(6,12))
            elif u==a:indices=[0,1,2,6,7,8,9,10,11,3,4,5]
            else:indices=[6,7,8,0,1,2,3,4,5,9,10,11]
            cycle.extend((a,b,i) for i in indices)
    return graph,cycle

def valid_cycle(graph,cycle):
    return len(cycle)==len(set(cycle))==len(graph) and set(cycle)==set(graph) and all(v in
graph[u] for u,v in zip(cycle,cycle[1:]+cycle[:1]))

if __name__=='__main__':
    g,c=cover_to_cycle([0,1,2],[(0,1),(1,2)],[1])
    print('vertices',len(g),'cycle',c,'valid',valid_cycle(g,c))
```

NP-completeness solutions (continued)

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study